

Software Lifecycle Management



Fachbereich Elektrotechnik
und Technische Informatik

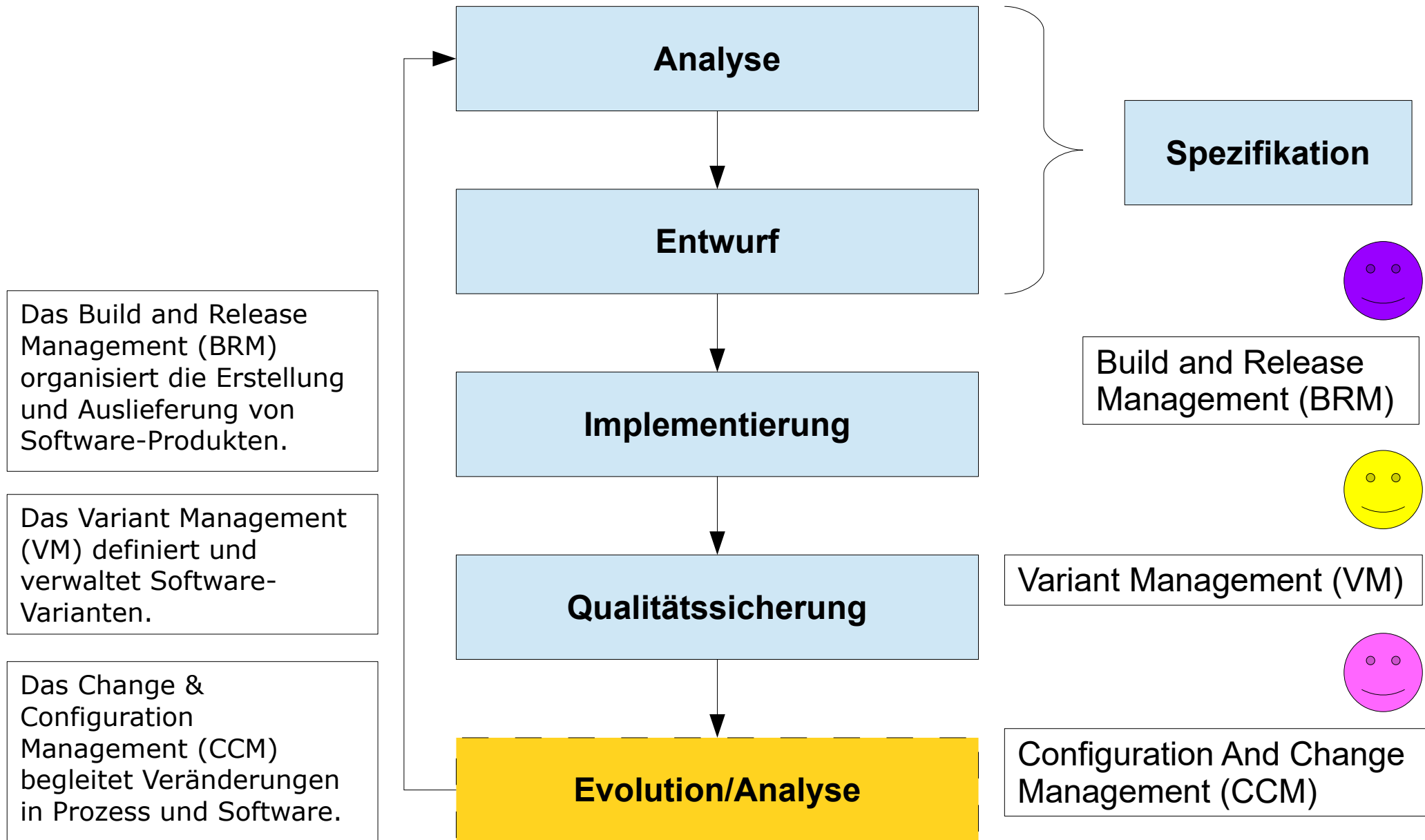
Dr. rer. nat. Nils Beckmann
nils.beckmann@th-owl.de

Evolution

Themen

- Software-Evolution
 - Definitionen
 - Änderungen: (CCM)
 - Wartung, Altsysteme, Weiterentwicklung
 - Formaler Änderungsprozess
 - Varianten: (VM)
 - Typen und Arten
 - Umsetzung einer neuen Variante
 - Builds: (BRM)
 - Build-Server, Nightly Build, Stabilität
 - Releasewechselplan

Erinnerung: Softwareprozessphasen



Definitionen

- Das **CCM** (Configuration and Change Management) organisiert Änderungsanträge ("Request for Change", RfC) sowie -pläne und Änderungsberichte ("Change Record", RC) sowie die entsprechenden Genehmigungsprozesse.
- Das **BRM** (Build and Release Management) ist das "ausführende Organ" des CCM und überwacht die Umsetzung und Durchführung.
- "Das **VM** (Variant Management) umfasst alle Steuerungsvorgänge zur Optimierung der Variantenvielfalt (der Software) und zur Beherrschung variantenreicher Produktspektren"
(nach Menge)

Definitionen

- Variante ("Was?")
 - "Eine Variante eines technischen Systems ist ein anderes technisches System gleichen Zwecks, das sich in mindestens einer Beziehung oder einem Element unterscheidet, oder ein Element unterscheidet sich von einem anderen Element in mindestens einer Eigenschaft."
(nach *H. Franke*, "Das QM-System [...]")
 - = ein ähnliches System oder Produkt
- Version
 - Entwicklungsstadium einer Software(-Variante)
- Eine Variante ↔ Viele Versionen

Definitionen

- Build ("Bauen", "Erstellung")
 - Technisch-Informatischer Prozess zur Erstellung eines lauffähigen Software-Produkts
- Release ("Veröffentlichung")
 - *Definierte Version* einer lauffähigen Software(-Variante) am Ende eines spezifizierten Build-Prozesses
 - Ziel: Bereitstellung an Dritte

- i.d.R. mehr Versionen als Releases oder Varianten
- Variante/Release besteht aus bestimmter Konfiguration

Definitionen

„Lehrbuch der Software-Technik – Management“, Balzert, 1995, S. 234 f.

- Konfiguration ("Wie?")
 - "Eine Software-Konfiguration ist eine benannte und formal freigegebene Menge von Software-Elementen, mit den jeweils gültigen Versionsangaben, die zu einem bestimmten Zeitpunkt im Produktlebenszyklus in ihrer Wirkungsweise und ihren Schnittstellen aufeinander abgestimmt sind und gemeinsam eine vorgesehene Aufgaben erfüllen sollen."
 - = Individuelle Zusammenstellung
- Software-Element
 - "Ein Software-Element ist jeder identifizierbare und maschinenlesbare Bestandteil des entstehenden Produkts oder der entstehenden Produktlinie."

Hypothesen der Veränderung

- 1) Kontinuierliche Veränderung: In einer realen Umgebung verliert das Programm kontinuierlich an Nutzen, da sich die Umgebung ändert.
- 2) Zunehmende Komplexität: Durch Änderungen wird ein Programm immer komplexer – entsprechend müssen Ressourcen zur Vereinfachung des Programms eingesetzt werden.
- 3) Konstanz der Systemeigenschaften: Ein System, was zu Beginn schon viele Probleme und Fehler hat wird auch in einem späteren Stadium viele Probleme und Fehler haben.

➤ **Veränderung** zwingend notwendiger **Prozess**!

Wartung und Weiterentwicklung

- *Wartung*

- „Die Veränderung eines Softwareprodukts nach dessen Auslieferung, um Fehler zu beheben, Performanz oder andere Attribute zu verbessern oder Anpassungen an die veränderte Umgebung vorzunehmen.“

(Definition gemäß IEEE 610.12-1990)

- "Korrektive Wartung" (Fehler beseitigen)
- "Perfektionierende Wartung" (Verbesserung)
- "Adaptive Wartung" (Anpassung an Wandel)

➤ Neue Versionen, Konfigurationen

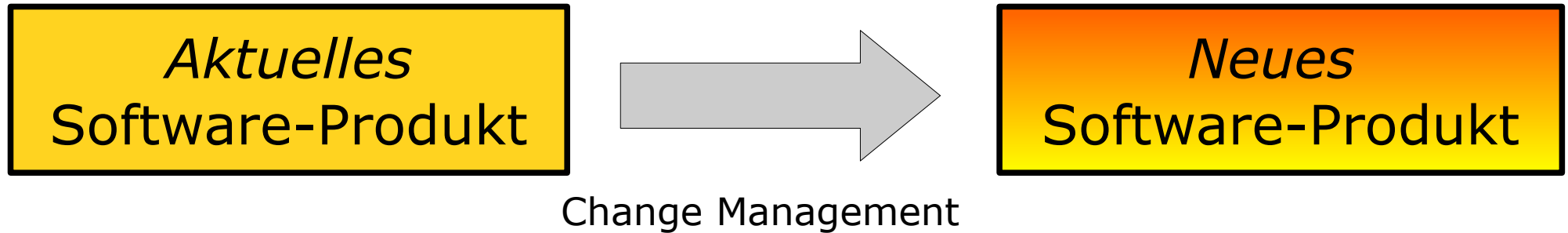
- *Weiterentwicklung*

- Version, Konfiguration, Variante, Release, Produkt

Möglichkeiten der Veränderung

- Fehlerbehebung
 - Bugs, Defects, Failures, Errors, Patches, ...
- Software-Anpassung
 - Refactoring (Umstrukturierung), Algorithmen anpassen, Kohäsion erhöhen, Code-Duplizierung entfernen, Plattform ändern, "Aufräumen", Dokumentationen bearbeiten, ...
- Funktionalitäten
 - Hinzufügen, Ändern, Entfernen

Software-Änderung: Vorher und Nachher



- **Analyse:**
Probleme, Möglichkeiten,
Risiken, Machbarkeit, SWOT-
Analyse, Portfolio-Analyse, ...
- **Änderung**
Prozesse, Antrag (RfC),
Weiterentwicklung, ...

- **Neues**
Quelltext, Begleitdokumente,
Dokumentation, ...
- **Änderungshistorie:**
Nachverfolgbarkeit, Audits,
Verifizierung und Validierung
(V&V), Change Record (CR), ...

Weiterentwicklungsprozesse

- Formen der Weiterentwicklung:
 - A) Fundamentaler Änderungsprozess
 - B) Systemweiterentwicklungsprozess
 - C) Änderungsimplementierung
 - D) Notreparaturprozess
 - E) Forward-Engineering
 - F) Software-Reengineering
- u.A.

A) Fundamentaler Änderungsprozess

- Grundlegender Prozess bei Veränderungen

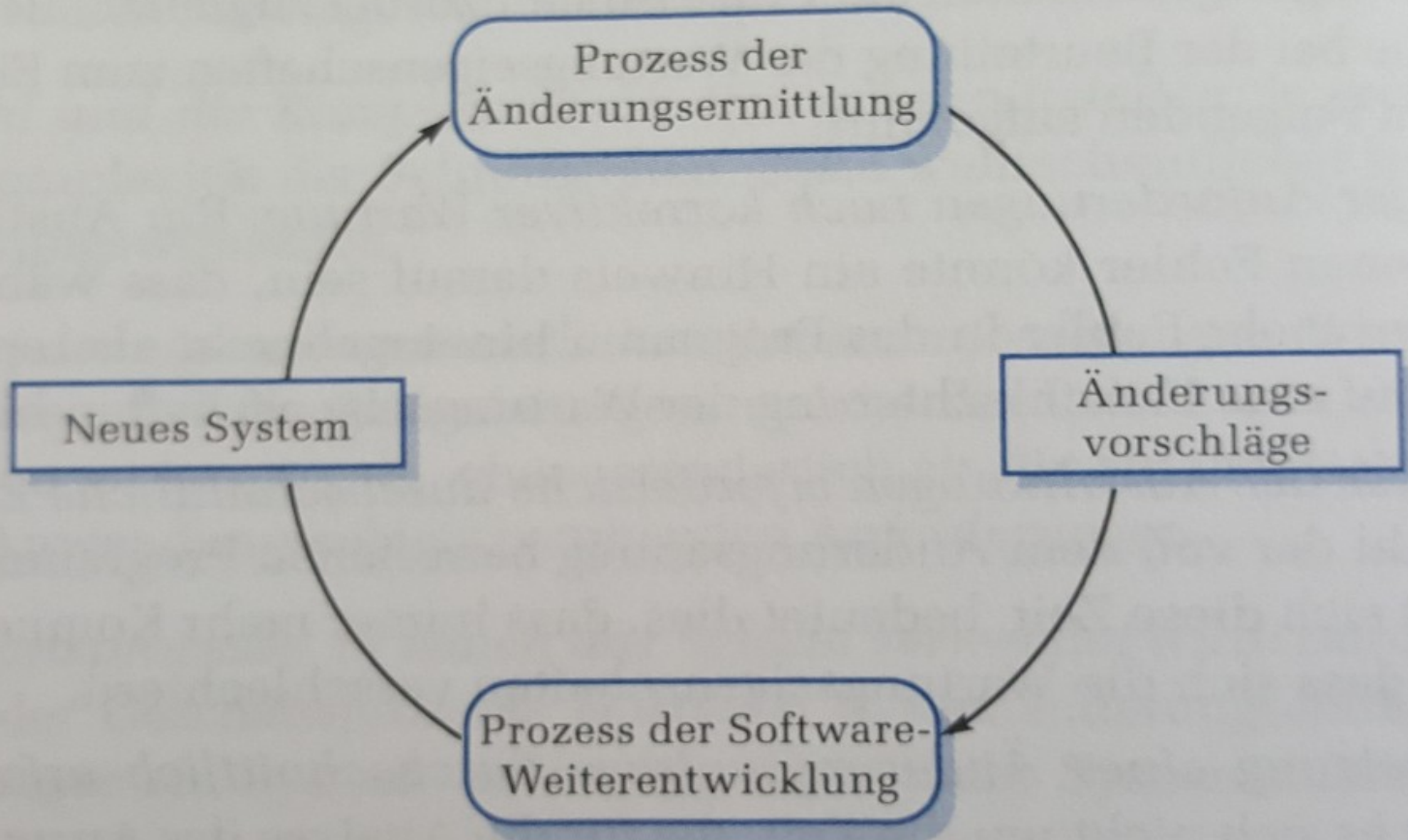
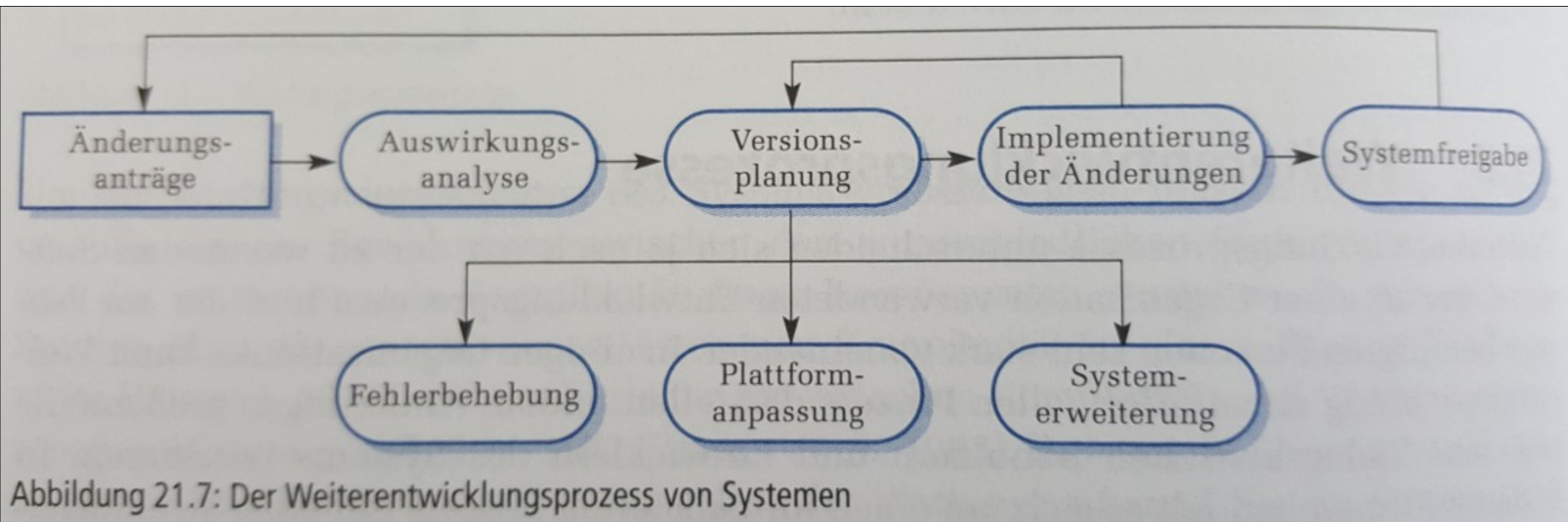


Abbildung 21.6: Prozesse des Erkennens von Änderungen und der Weiterentwicklung

B)

Systemweiterentwicklungsprozess

- Berücksichtigung von Eigenschaften des Gesamtsystems und möglichen Auswirkungen einer Veränderung



C) Änderungsimplementierung

- Berücksichtigung:
Änderung ↔ Bestehende Anforderungen

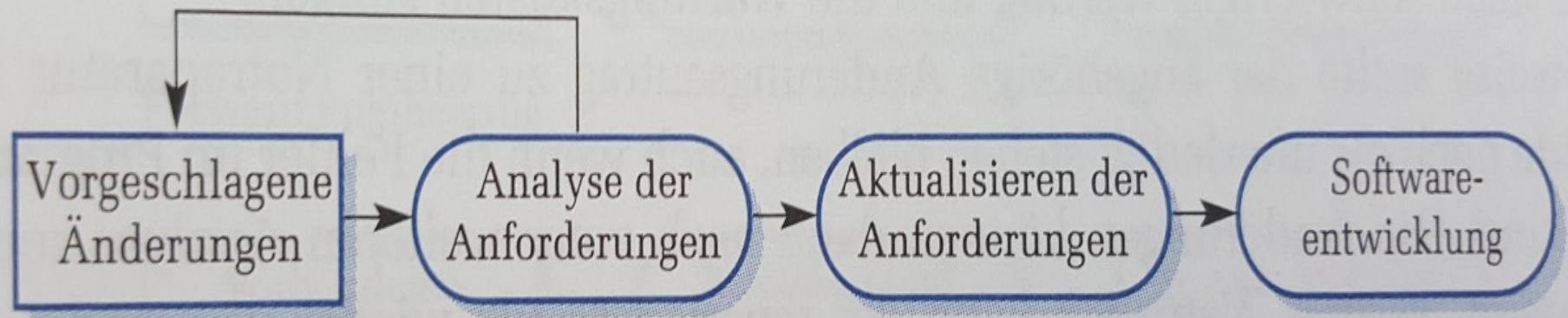
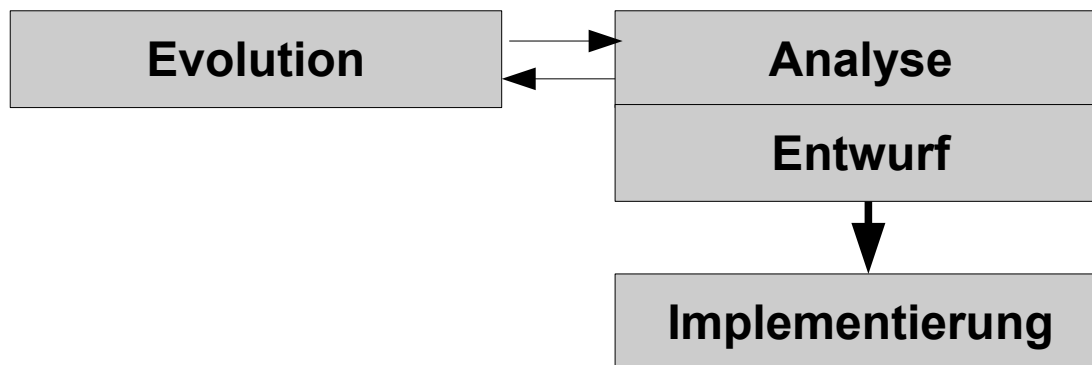


Abbildung 21.8: Implementierung von Änderungen



D) Notreparaturprozess

- Nur für sehr dringende Änderungen:
 - Bei ernsthaften Systemfehlern, ohne deren Behebung der Betrieb ruht
 - (Unvorhergesehene) Äußere Umgebungsänderungen stören Normalbetrieb
 - Neue Gesetze, Mitbewerber, etc.

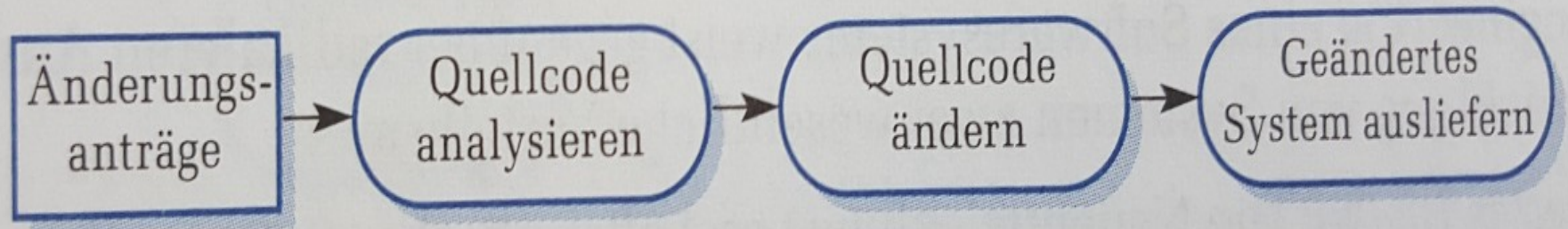


Abbildung 21.9: Der Notreparaturprozess

E) Re- vs. Forward-Engineering

- **Forward Engineering:** Neues System entwickeln
- **Re-Engineering:** Altes System weiterentwickeln
→ Kann kostengünstiger sein!

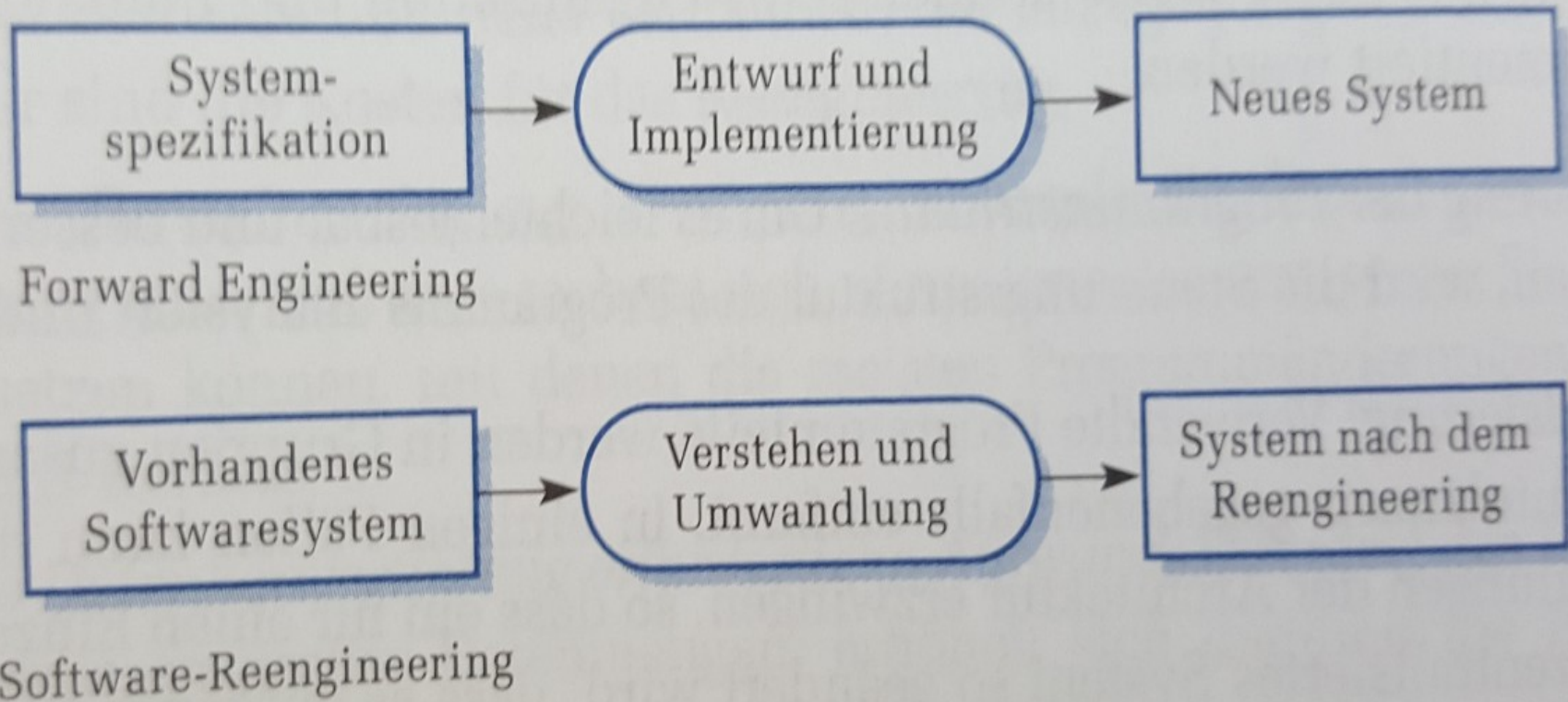


Abbildung 21.10: Forward Engineering und Reengineering

F) Re-Engineering

- Neugestaltung eines bestehenden Systems
- Start ist altes System - nicht Spezifikation!

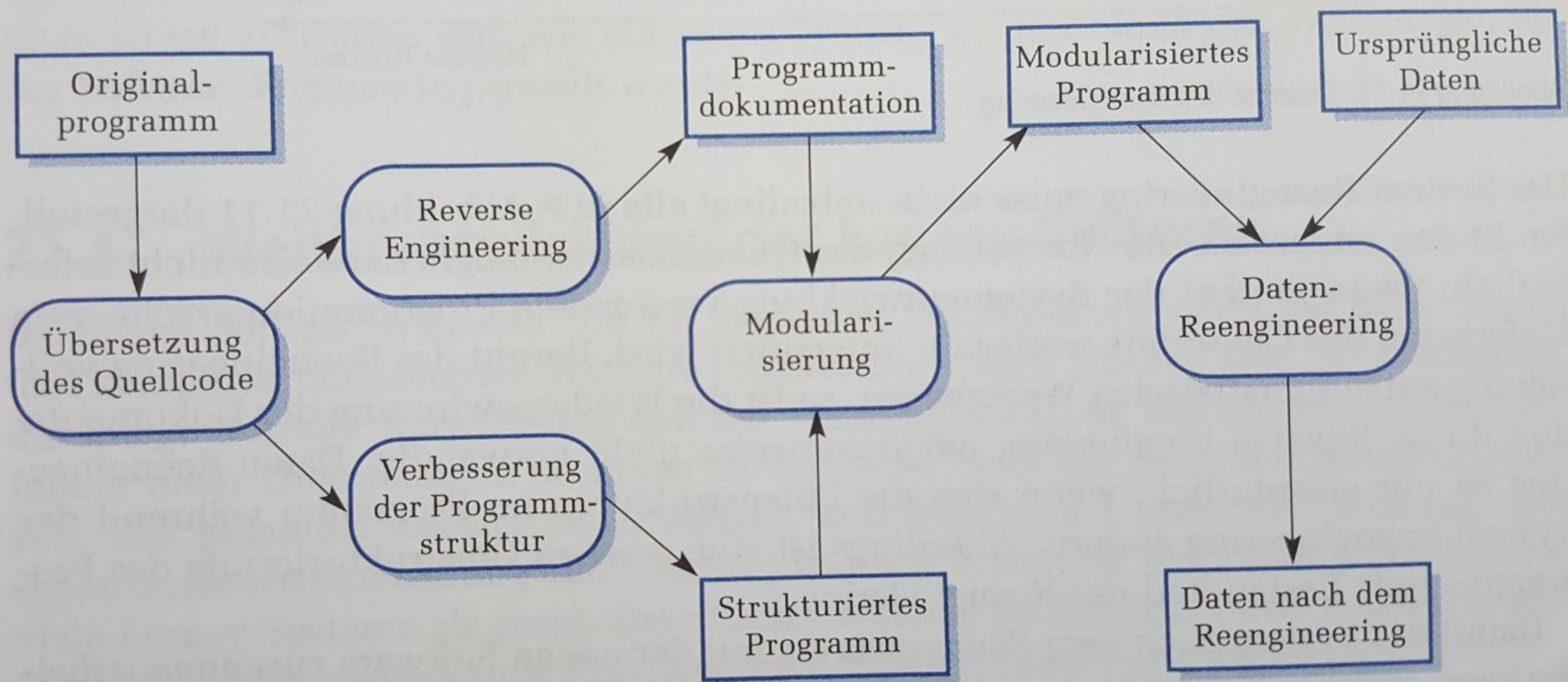


Abbildung 21.11: Der Reengineering-Prozess

Umgang mit Altsystemen

- Altsysteme ("legacy systems") sind historisch gewachsen und werden weiter betrieben, obwohl sie überholt sind und ein neues/anderes System besseren Nutzen erzielen kann

- Möglichkeiten:

„Software Engineering“, Sommerville, S. 546

- 1) Ausmustern und nicht ersetzen
- 2) Unverändert weiter betreiben ohne Wartung
- 3) Unverändert lassen und regelmäßig warten
(= wenig Änderungswünsche umsetzen)
- 4) Per Re-Engineering sanieren: Erhöhung der Wartbarkeit
- 5) Teile des Systems komplett ersetzen
- 6) Das System komplett durch ein neues ersetzen
- 7) Nur die Teile weiterbetreiben, die (noch) funktionieren

Umgang mit Altsystemen

- "Portfolio-Analyse":
 - Beispiel: Systemqualität vs. Geschäftswert

	Qualität hoch	Qualität niedrig
Wert Hoch	3) Weiterbetreiben mit Wartung (Weger hoher Qualität Sanierung nicht notwendig) Ggf. 5) Systemteile niedriger Qualität ersetzen	4) Sanieren mit Re- Engineering Oder 6) Komplette ersetzen
Wert Niedrig	2) und 7) Weiterbetreiben, solange Wartung günstig bleibt Sonst 1) Ausmustern	7) Nur Weiterbetreiben, was noch läuft Oder 1) Ausmustern

Request for Change (RfC)

- Der **Änderungsantrag** ("Request for Change", RfC) ist das grundlegende Werkzeug für eine **Änderung** durch einen **Änderungsprozess**
 - Prüfung
 - Planung
 - Durchführung
 - Nachverfolgung
- Zur Vorbereitung Informationen sammeln:
 - Bisherige Aktivitäten, Diskussionen, Beschreibungen, Analysen, Dokumentationen, Entscheidungen

Request for Change (RfC)

- Erstellung des Änderungsantrags:
 - Registrierung: Benötigte Informationen sammeln, Entscheidungsfindung vorbereiten
 - Klassifizierung: Umfang, Risikoabschätzung, Auswirkungen
 - Planung: Terminplan erstellen (z.B. Balkenplan), Ausführungsplan erstellen, Kosten kalkulieren
- → Dann erfolgt Einreichung des Änderungsantrags zur Genehmigung und Durchführung.

Request for Change (RfC)

```
Anforderungsänderung durch Ausfüllen eines Änderungsantrags
Änderungsantrag analysieren
if Änderung sinnvoll then
    Überlegen, wie Änderung implementiert werden kann
    Änderungskosten beurteilen
    Änderungsantrag in Datenbank festhalten
    Antrag beim Änderungsgremium einreichen
    if Änderung akzeptiert then
        repeat
            Änderungen an Software vornehmen
            Änderungen protokollieren und mit zugehörigen Anträgen verknüpfen
            geänderte Software zur Qualitätsprüfung einreichen
        until Softwarequalität akzeptabel
        neue Systemversion erstellen
    else
        Änderungsantrag ablehnen
else
    Änderungsantrag ablehnen
```

Abbildung 29.3: Ablauf des Änderungsmanagements

Änderungsantrag: Inhalte

- Allgemeines: Projektname, Release, Antragsteller, ID, Datum
- Beschreibung der beantragten Änderung
- Änderungsanalyse Bearbeiter, Datum, betroffene/verknüpfte Komponenten, Beurteilung der Änderung, Änderungspriorität, Möglichkeiten und Risiken, Auswirkungen der Nicht-Umsetzung, Gegenmaßnahmen
- Schätzungen für die Umsetzung hinsichtlich Zeit, Kosten, Personal, sonstige Ressourcen (sowie woher diese kommen)
- Jeweilige Datumsangaben zu Entscheidungen: Vorlagedatum und Entscheidungsdatum (jeweils bei CCM, TQM). Durchführen? Wie durchführen? Für welches Release? Mit welcher Konfiguration? Woher Ressourcen? Budgetfreigabe?
- Zuständigkeiten: Implementierung, Dokumentation
- Weiteres: Anhänge, Kommentare, Referenzen, Analyseergebnisse

Change Record (CR)

- Der **Änderungsbericht** ("Change Record", CR) bezieht sich immer eindeutig auf einen **Änderungsantrag** (RfC)
 - Enthält laufend aktualisierte Dokumentation zum Verlauf der Änderung
 - Verläuft die Änderung planmäßig?
 - Welche Schwierigkeiten treten auf?
 - Welche Gegenmaßnahmen werden ergriffen?
 - Referenziert entsprechende bisherige Überlegungen und Dokumente (RfC, usw.)
 - Fazit bei Abschluss der Änderungsdurchführung
 - Nach Plan oder nicht? Abweichungen?

Weitere Dokumente des Änderungsmanagement

- Änderungsantrag: Wunsch nach Änderung
- Änderungsvorschlag: Technische und wirtschaftliche Bewertung der Änderung
- Änderungsauftrag: Erfordert Genehmigungsverfahren, enthält Lösungsweg, enthält Anforderungen
- Änderungsmitteilung: Beschreibung der durchgeführten Änderungen
- Änderungsstatusliste: Liste aller Änderungsanträge mit Status für den Überblick
- Problemmeldung: Information über ein bestehendes Problem
- Problembericht: ("Problem Record") Information über Probleme während eines Prozesses
- u.A.

Abgrenzung CCM & VM

- CCM = "Change and Configuration Management"
- VM = "Variant Management"
- Konfigurations-Management: Wie?
 - ... ist der aktuelle Zustand?
 - ... wird produziert?
 - ... sind die Einzelteile arrangiert?
- Varianten-Management: Was?
 - ... wird produziert?
 - ... wird angeboten?

Konfigurationsmanagement

- Zentrales Dokument:
 - Konfigurations-Identifikationsdokument (KID)
oder "Konfigurationshierarchie"
oder "Elementstrukturplan"
 - Definiert für eine **Konfiguration**, welche **Software-Elemente** zu ihr gehören
 - Enthält entsprechenden Hinweise (auf Quelltext, Dokumente, genutzte Compiler, Schnittstellen, Plattformen, usw.)
 - Konfigurationen können über Versionsverwaltungstools archiviert und abgerufen werden

Tools: Versionsverwaltung

- **Kommerziell:**

- Team Foundation Server
<https://visualstudio.microsoft.com/de/tfs/>
- AlienBrain <http://www.alienbrain.com/>
- Perforce <https://www.perforce.com/>

- **Frei verfügbar:**

- Subversion (SVN) <https://subversion.apache.org/>
- GIT <https://git-scm.com/>
- MediaWiki <https://www.mediawiki.org/wiki/MediaWiki/de>
- <https://de.wikipedia.org/wiki/Versionsverwaltung>
- u. v. m.

Motivation für Varianten-Management

- Viele Kunden $\leftrightarrow$ Viele individuelle Produktwünsche
- Diese Produkte ähneln sich oft nur in Details
 - Viele Grundfunktionen in allen Produkten
- Alle Produkte parallel entwickeln $\Rightarrow$ Großer Mehraufwand (Zeit, Personal, Kosten)
- VM verringert Mehraufwand $\rightarrow$ Vollständige Parallelentwicklung wird reduziert durch andere Konzepte
- Beispiele für Variantenangebote: Plattformkonzepte mit Basisfunktionen und Add-On's, Variantentypen, Baukastensysteme, Module, Plug-In's, usw.

Variantentyp: Standardvarianten

- Kernfunktionen der Software identisch
⇒ Gleiches Produkt
- Varianten:
 - Mit anderen Sprachen
 - Mit anderer GUI
 - Mit anderer optischer Repräsentation
 - Mit anderer Datenhaltung
 - usw.

Variantentyp: Leistungsklassen

- Gleiches Produkt mit gleichen Funktionen
- Andere Leistungsklassen:
 - Standalone, Server-Client, Cloud-Basiert, mit AdHoc-Netzwerk oder WLAN-Verbindung
 - Höhere Performance, Geschwindigkeit, Reaktionszeit
 - Geringere Rechenzeit, weniger Speicherbedarf
 - Andere Nicht-Funktionale Anforderungen:
 - Höhere Zuverlässigkeit, Höhere Stabilität, usw.

Variantentyp: Funktionsmerkmale

- Gleiches Produkt
- Unterschiedliche Funktionen, Funktionsmerkmale:
 - Allgemein: Detailgrad oder Auswahlmöglichkeiten
 - Auflösung der Darstellung, Berechnungstiefe, Einstellungsmöglichkeiten
 - Konfigurierbarkeit
 - Personalisierung (Bezug zum Nutzer)
 - Organisatorische Handhabung (Management) – Möglichkeiten der zentralen Administration
 - Algorithmenvarianten
 - Komfortmerkmale, Bessere Bedienbarkeit, usw.

Möglichkeiten für Variantenarten

- Variantentypen: Standardvarianten, Leistungsklassen, Funktionsmerkmale
- Basis-Funktionen + Add-On's: Das Basis-Softwareprodukt muss erworben werden
- Modularisierung: Programmteile werden getrennt vertrieben
- Plattformkonzept: Kunde erhält erweiterbaren, zentralen Zugang zu Funktionalitäten
- Plug-In-Konzept: Weitere Unter-Funktionalitäten und Verfeinerungen (z.B. aktualisierbare DLL's) können später erworben werden

Möglichkeiten für Variantenarten

- Baukastenprinzip: Nutzer stellt sich selbst Produkt zusammen
- Upgrades: Software wird für den Kunden direkt laufend verfeinert oder erweitert
- Service-Vertrag: Nutzer oder Kunde erhält Funktionsupgrades nach individuellem Wunsch
- Lizenz: Testphase, Shareware, Eingeschränkte Funktionalität, etc.
- u.A.

Umsetzung einer Variante

- Jede Variante erfordert:
 - Projektumfeld (intern)
 - Produktmarkt (extern)
- Aufgaben:
 - Marktrecherche, bestehende Varianten analysieren, Prüfen auf Kanibalisierung, Planung einer Variante, Konsolidierung der Variante im Unternehmen, Migration der Variante in den Markt
- Jede Variante hat Relevanz im Bereich Management und Vertrieb

Varianten Nebenbedingungen

- Kosten und Aufwand:
 - Verwaltung, Produktmanagement, versteckte Kosten durch komplexe Abhängigkeiten, Pflege (Wartung, Service, Helpdesk, Installation, etc.), ...
- Risiken:
 - Beherrschbarkeit, Vollständige Abdeckung über Lebenszyklus, Zusatzaufwendungen, Kosten, Konsolidierung intern und extern, Kanibalisierung, usw.
 - Nur Varianten mit Marktpotential sollten umgesetzt werden

Build and Release Management

- Das "Build und Release Management" (BRM) ist das "ausführende Organ" des "Configuration and Change Management" (CCM)
 - CCM: Änderungsanträge und Änderungspläne
 - BRM: Überwachung der Umsetzung
 - Durchführung, Sicherstellung der Ressourcen, Einhaltung von Kosten und Zeiten, Qualitätssicherung, Koordination des Teams, Verfolgung und Übersicht der Build-Versionen und Realeses
 - Stellt Durchführung von Builds und Erstellung von Releases sicher

Build-Arten

- "Overall":
 - Build über alle Teile und Module eines Software-Produkts
- "Part":
 - Build über Teile des Software-Produkts oder Modulbereiche
- "Daily":
 - Täglicher Build-Prozess (i.d.R. als "Nightly Build")
 - z.B. automatisiert über "Build-Server"
 - Gängige Praxis in vielen größeren Projekten und Gemeinschaftsprojekten

Definition "Versionsstabil"

- Die Version der Software ändert sich nicht mehr
 - Keine zusätzlichen Funktionen werden mehr eingebaut
 - Keine (größeren) Bugs müssen mehr repariert werden
 - Keine größeren Refactorings oder Umbauten hinsichtlich Datentypen, Datenbanken, usw. geschehen mehr
- Die Software interagiert mit erstellten Daten aus früheren Versionen sicher und absturzfrei

Definition "Produktionsstabil"

- Das Release des Software-Produkts ändert sich nicht mehr
 - Das Release ist versionsstabil
 - Es gibt keine Änderungen mehr an...
 - ...Darstellung
 - ...Algorithmen
 - ...Datenhaltung
 - ...usw.
- $\Rightarrow$ Produktionsstabil $>$ Versionsstabil

Release-Arten

- "Release-Art" / "Software-Entwicklungsstadium":

1. Alpha

2. Beta

3. Release Candidate (RC)

4. Stable

Alpha
Beta
RC
Stable

- Weitere (u.A.):

- "Pre-Alpha" (Entwicklervorschau)
- "Perpetual Beta" / "Continuous Integration" (genutzt in XP, quasi niemals "fertig")
- "Closed Beta"
- "Public Beta"

Release-Art: "Alpha"

Alpha

- Software noch nicht ausgereift
- Enthält ggf. Bugs, Defects, Probleme, usw.
- Bereitstellung u.U. intern:
 - Für Qualitätssicherung (QS) und Testen
 - Für Mit-Entwickler
 - Ggf. für Projekt-Partner zur Demonstration neuer Features
- Keine Weitergabe an Kunden
- Meist nicht Versionsstabil

Release-Art: "Beta"

Beta

- Reiferes Software-Produkt als "Alpha"
- Wurde bereits getestet
- Typische Bereitstellung für...
 - Qualitätssicherung (QS) und zum Testen
 - Ggf. an ausgewählte Beta-Kunden
 - Ggf. an (externe) Beta-Tester
- Enthält alle neuesten Features, Bug-Fixes, usw.
- Benötigt ggf. noch weitere Tests
- Keine Weitergabe an Kunden
- Meist nicht Versionsstabil

Release-Art: "Release Candidate"

RC

- Release kurz vor der Auslieferung an Kunden
- Enthält keine bekannten Fehler mehr
- Oft müssen noch eine Reihe von Kleinigkeiten gelöst werden:
 - Dokumentation überarbeiten, Menüführung optimieren, Grafische Darstellung feinjustieren, ...
- Eingeschränkte Weitergabe an Kunden, wird Kunden ggf. als Release bereits angeboten
- Erfährt noch mehrere Iterationen (RC1, RC2, ...)
- Ist bereits stabil verwendbar
- Ist Versionsstabil, noch nicht Produktionsstabil

Release-Art: "Stable"

Stable

- Abgeschlossene Produktversion
- Ist Versionsstabil und Produktionsstabil
- Wird Kunden angeboten
- Wird bis zum nächsten Stable Release mit Bug-Fixes und Patches versehen
- Verkaufsversion

Aspekte jedes neuen Releases

- Jedes neue Release erfordert ein Projektumfeld:
 - Generelle Planung
 - Was macht Wer Wann Warum?
 - Fragen zum Release
 - Neuinstallation? Überinstallieren? Sind Besonderheiten zu beachten? Upgrade? Aktualisieren? Migration? ...
 - Umfeld beachten
 - Was ist neben dem eigentlichen Software-Release zu beachten?

Releases aus Managersicht

- Releases in Sprint Meetings berücksichtigen
Effektiv am Quelltext arbeiten und keine doppelte Arbeit, ggf. Refactoring/Backlogänderungen anregen
- Teams frühzeitig einbeziehen
Für fachliche Rückmeldung und um das Team "mitzunehmen"
- Risiken offen ansprechen
Für Leadership auf allen Ebenen und Vorbeugestrategien
- Kaufmännische Themen beleuchten
Wirtschaftlichkeit prüfen und sicherstellen
- Wertschöpfung und Risiken für Kunden bedenken
Mehrwert und Interesse des Kunden sicherstellen
- Verantwortung verteilen
Durch Aufgabenteilung effektiver vorgehen
- Motivation für gemeinsame Zielerreichung
Nicht "gegen" sondern "mit" dem Team arbeiten

Bedarfe prüfen für jedes neue Release

- Fragen bezüglich Kosten und Zeit:
 - Gibt es Kosten durch eine Downtime oder Nicht-Verfügbarkeit des Systems? Muss die Produktion stoppen? Welche Produktionsausfälle gibt es dadurch? Muss eine Testphase eingeplant werden? Welche Kosten entstehen durch Umstellungsprobleme? Müssen Schulungen durchgeführt werden? Ist das neue Release genauso intuitiv bedienbar wie das alte? Sind alte Funktionen umgestellt worden? Müssen neue Dienstleistungen geschaffen werden (Verwaltung, HelpDesk, Service, Support, Pflege, Wartung, Kundentelefon, usw.)? ...
- Versteckte Kosten und Zeitbedarfe durch mangelnde Kommunikation oder weniger optimale Zusammenarbeit berücksichtigen

Nightly Build und Build-Server

- Der "Nightly Build" ist ein "Daily Build"
 - Automatisiert durchgeführt (i.d.R. nachts)
- Zum Build-Prozess kann gehören:
 - Kompilieren, Linken (der Bibliotheken)
 - Unit-Tests
 - Regressionstests (automatisierte Wiederholungstests)
 - Berechnung von (aufwändigen) Software-Qualitätsmetriken
 - Installation in die Zielumgebung ("Deployment")
 - usw.
- Für die Nutzung und Weitergabe ist generell ist ein "Stable"-Release einem "Nightly Build" vorzuziehen

Tools: Build-Server und Testmanagement

- **Kommerziell:**

- Jenkins <https://jenkins.io/>
- Atlassian Bamboo
<https://www.atlassian.com/software/bamboo>

- **Frei verfügbar:**

- BuildBot <https://buildbot.net/>
- Travis CI <https://travis-ci.org/>
- JetBrains TeamCity
<https://www.jetbrains.com/teamcity/>

- u. v. m.

Releasewechselplan

- Umfänglicher Plan um einen Release-Wechsel zu strukturieren und zu überwachen, enthält die folgenden Informationen:
 - Allgemeine Informationen (Projekt, Produkt, Ersteller, ...)
 - Prognosen zu Chancen, Risiken, Marktpotential, usw.
 - Spezifikation des neuen Releases
 - Gemeinsamkeiten und Abgrenzung zum vorigen Release
 - Genehmigungsschritte
 - Definition der Arbeitspakete für das neue Release
 - Schätzungen (Kosten, Zeit, Ressourcenbedarfe, ...)
 - Strukturierte Planungen für die Umsetzung (Balkenplan, ...)
 - Einplanungen von Ressourcen, die nicht direkt mit der Software-Entwicklung zu tun haben (Schulungen, Auslieferung, HelpDesk, usw.)

Vorteile durch BRM

- Software-Qualität kann erhöht werden
- Das Team hat eine Leitlinie ("Roter Faden")
 - Teammitglieder werden einbezogen und tragen Verantwortung
 - Schnittstellen im Team werden transparent und bilden keine Barrieren
- Nur Releases mit ausreichender Qualitätssicherung (Testqualität) werden weitergegeben
- Der Releasewechsel wird transparent [Was? Wer? Wann? Warum?]
 - Kontinuierlicher Prozess über verschiedene Schritte/Phasen entsteht

Zusammenfassung

- Die Weiterentwicklung (Evolution) von Software ist essentieller und notwendiger Bestandteil des Software-Entwicklungsprozesses
 - Wichtiges Werkzeug des CCM ist Änderungsantrag
 - Je nach Software sind unterschiedliche Prozesse der Weiterentwicklung geeignet
 - VM reduziert Mehraufwand/Parallelarbeit und Produktpalette wird transparent und strukturiert
 - Nach Innen: Modulare Arbeit an Varianten
 - Nach Außen: Individuelle Varianten für Kunden
 - Das BRM ist ausführendes Organ des CCM
 - Überwacht Änderungen, Entwicklungen, Releases

Fragen?